

PlastiWatch — IMU Data Collector

Embedded firmware for the **Seeed Studio XIAO ESP32C3** that collects 6-axis motion data from an **MPU6050** sensor, labels it with an activity name, and uploads it wirelessly to **Edge Impulse** for TinyML model training.

Physical Motion → MPU6050 → ESP32C3 → WiFi → Edge Impulse Training Dataset

Table of Contents

1. [Quick Start](#)
 2. [What You Need](#)
 3. [Hardware Wiring](#)
 4. [Software Setup](#)
 5. [Configure the Firmware](#)
 6. [User-Configurable Parameters](#)
 7. [Build and Flash](#)
 8. [Collecting Your First Data](#)
 9. [Understanding the Display](#)
 10. [Serial Monitor Commands](#)
 11. [LED Status Guide](#)
 12. [Troubleshooting](#)
 13. [Code Deep Dive](#)
-

Quick Start

Follow these five steps to go from unboxed hardware to uploading your first dataset:

```
Step 1 → Wire up the hardware          (see Hardware Wiring)
Step 2 → Edit Config.h                 (WiFi credentials + Edge Impulse API
key)
Step 3 → Build and flash                (one click in VS Code / PlatformIO)
Step 4 → Power on → place flat → press button to calibrate IMU
Step 5 → Press button to pick label, long-press (2 s) to start recording
```

That's it. The device calibrates itself, then records 1000 IMU samples (10 s), uploads them, and immediately starts the next window — continuously collecting data until you press the button to stop.

What You Need

Hardware

Item	Notes
Seeed Studio XIAO ESP32C3	The microcontroller board
MPU6050 IMU module	6-axis (accelerometer + gyroscope)
SSD1306 OLED display (128×64)	I2C interface
Tactile push button	Momentary — short press cycles label; long press starts continuous recording session
Jumper wires	Female-to-female for breadboard
Breadboard	Optional but recommended
USB-C data cable	Must support data — charge-only cables won't work

Software

Tool	Link
VS Code	https://code.visualstudio.com
PlatformIO IDE extension	Search "PlatformIO IDE" in VS Code Extensions
Edge Impulse account (free)	https://studio.edgeimpulse.com

Driver note — Windows: If the XIAO's serial port does not appear, install the **CH343** driver. Search for **CH343SER.EXE** on the Seeed Wiki and run the installer.

Hardware Wiring

Both the MPU6050 and the OLED share the **same I2C bus** (SDA + SCL lines). You can connect them in parallel — all three components share the same two wires. The label button requires no external resistor because the firmware enables the ESP32-C3's **internal pull-up resistor** on GPIO3.

XIAO ESP32C3

3.3V ————— VCC

GND ————— GND

D4 (GPIO6 / SDA) ————— SDA

D5 (GPIO7 / SCL) ————— SCL

MPU6050

VCC ————— VCC

GND ————— GND

SDA ————— SDA

SCL ————— SCL

AD0 — GND (sets I2C address to 0x68)

OLED SSD1306

VCC ————— VCC

GND ————— GND

SDA ————— SDA

SCL ————— SCL

Label / record button (no external resistor needed)

D3 (GPIO3) ————— Button leg A

GND ————— Button leg B

When button is OPEN: GPIO3 is pulled HIGH by internal resistor → not

```
pressed
When button is PRESSED: GPIO3 is pulled LOW by GND → press detected
```

Pin Reference Table

XIAO ESP32C3 Pin	Connects to
3.3V	MPU6050 VCC and OLED VCC
GND	MPU6050 GND and OLED GND and button leg B
D3 (GPIO3)	Button leg A — short press cycles label; hold 2 s starts continuous session; press during recording stops session
D4 (GPIO6)	MPU6050 SDA and OLED SDA
D5 (GPIO7)	MPU6050 SCL and OLED SCL
(MPU6050 AD0)	GND — sets I2C address to 0x68

Power warning: The MPU6050 runs on **3.3 V**. Do not connect VCC to 5 V — it will damage the sensor.

Why no resistor? The ESP32-C3 has built-in pull-up resistors (~45 kΩ) on every GPIO pin. Enabling `INPUT_PULLUP` in firmware connects this resistor internally, so the pin reads HIGH when the button is open and LOW when pressed. Adding an external pull-up resistor is unnecessary.

Software Setup

1. Install VS Code and PlatformIO

- 1. Download and install [VS Code](#)
- 2. Open VS Code → Click the **Extensions** icon in the left sidebar (or `Ctrl+Shift+X`)
- 3. Search for **"PlatformIO IDE"** and click **Install**
- 4. Restart VS Code when prompted
- 5. Confirm the **PlatformIO alien-head icon** appears in the left sidebar

2. Open the Project

File → **Open Folder** → select the `plastiwatch_collector` folder.

PlatformIO detects `platformio.ini` automatically. You will see `seeed_xiao_esp32c3` in the bottom status bar.

3. Set Up an Edge Impulse Project

- 1. Create a free account at [studio.edgeimpulse.com](#)
- 2. Click **Create new project** and name it (e.g. "PlastiWatch Activity")
- 3. Go to **Dashboard** → **Keys** and copy your **API key** — you will need it next

Configure the Firmware

Open `src/config/Config.h` and update these four settings before flashing:

```
// — 1. Your WiFi network
-----
#define WIFI_SSID      "YourNetworkName"
#define WIFI_PASSWORD  "YourPassword"

// — 2. Your Edge Impulse API key (Dashboard → Keys)
-----
#define EI_API_KEY     "ei_XXXXXXXXXXXXXXXXXXXXXXXXXXXX"

// — 3. A unique name for this device
-----
#define EI_DEVICE_NAME "plastiwatch-001"

// — 4. Activity labels (must match your Edge Impulse project labels)
-----
static const char* const ACTIVITY_LABELS[NUM_ACTIVITY_LABELS] = {
    "standing",
    "walking",
    "running",
    "falling"
};
```

Important: The XIAO ESP32C3 supports **2.4 GHz Wi-Fi only**. Make sure your router broadcasts a 2.4 GHz network.

Everything else in `Config.h` has sensible defaults and can be left unchanged for your first session. See the section below for a full list of tuneable parameters.

User-Configurable Parameters

All parameters live in `src/config/Config.h`. The table below highlights the ones most commonly adjusted for a data collection session.

Essential — edit before flashing

Parameter	Default	Description
WIFI_SSID	"Hardware Community 2.4"	Your 2.4 GHz WiFi network name
WIFI_PASSWORD	(set in file)	Your WiFi password
EI_API_KEY	(set in file)	Edge Impulse API key (Dashboard → Keys)
EI_DEVICE_NAME	"plastiwatch-001"	Logical device name shown in Edge Impulse Studio

Parameter	Default	Description
ACTIVITY_LABELS[]	standing, walking, running, falling	Labels sent to Edge Impulse — must match your project exactly
NUM_ACTIVITY_LABELS	4	Number of labels in the array above

Data collection — tune to your experiment

Parameter	Default	Description
COLLECTION_DURATION_S	10	Length of each recording window in seconds. 10 s = 1000 samples at 100 Hz. Increase for longer windows; decrease for shorter ones. MAX_SAMPLES updates automatically.
SAMPLE_RATE_HZ	100	IMU sampling frequency (Hz). 100 Hz is a good balance for activity recognition.
RECORDING_START_DELAY_MS	2000	Countdown before recording begins (ms) after a long-press. A per-second countdown is shown on the OLED so you can get into position. Must be a multiple of 1000 ms.
IMU_CALIB_SAMPLES	200	Number of IMU samples averaged during the startup calibration. 200 × 10 ms = 2 s of calibration. Higher values give more accurate bias removal but take longer.

IMU sensor settings

Parameter	Default	Description
IMU_ACCEL_FS_G	2	Accelerometer full-scale range in g. Valid: 2, 4, 8, 16. Lower range = higher resolution.
IMU_GYRO_FS_DPS	250	Gyroscope full-scale range in deg/s. Valid: 250, 500, 1000, 2000.
IMU_DLPF_CFG	3	Digital Low-Pass Filter setting (0–6). 3 → 44 Hz accel / 42 Hz gyro cutoff.

Display and timing

Parameter	Default	Description
OLED_I2C_ADDRESS	0x3C	SSD1306 I2C address. Change to 0x3D if your display has the address jumper bridged.
OLED_RESULT_DWELL_MS	2500	How long the upload result screen stays visible (ms) before returning to IDLE.
BTN_LONG_PRESS_MS	2000	Hold duration (ms) required to trigger recording.

Parameter	Default	Description
<code>BTN_DEBOUNCE_MS</code>	<code>50</code>	Button debounce window (ms). Increase if you see spurious presses.

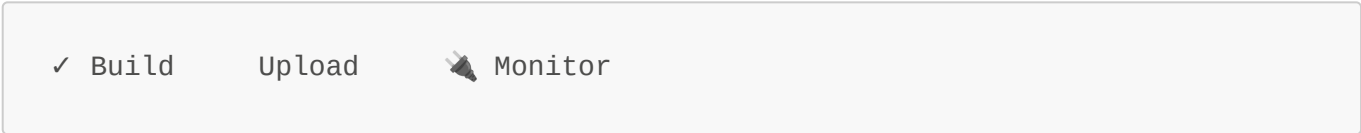
Network

Parameter	Default	Description
<code>WIFI_CONNECT_TIMEOUT_MS</code>	<code>15000</code>	Maximum time to wait for a WiFi connection (ms).
<code>EI_HTTP_TIMEOUT_MS</code>	<code>15000</code>	HTTP request timeout for uploads (ms).

Build and Flash

Option A — VS Code Status Bar (Recommended)

Look at the bottom status bar in VS Code for these buttons:



- 1. Click **✓ Build** to compile (watch the terminal — it should end with **SUCCESS**)
- 2. Plug in the XIAO via USB-C
- 3. Click **Upload** to flash the firmware
- 4. Click **🔌 Monitor** to open the serial console at 115200 baud

Option B — Command Line

```
# Compile (release build)
pio run -e seeed_xiao_esp32c3

# Compile and upload
pio run -e seeed_xiao_esp32c3 --target upload

# Open serial monitor
pio device monitor -b 115200
```

Build Environments

Environment	Debug logging	When to use
<code>seeed_xiao_esp32c3</code>	Off	Normal data collection sessions
<code>seeed_xiao_esp32c3_debug</code>	On (verbose)	Development and troubleshooting

If the Board Won't Enter Programming Mode

Hold **BOOT**, press and release **RESET**, then release **BOOT** to force the bootloader. Then run the upload command again.

Collecting Your First Data

Boot Sequence

On power-up, the OLED shows the splash screen and the serial monitor prints:

```
=====
PlastiWatch – IMU Data Collector  v1.0
=====
[INF][Main      ] Initialising IMU...
[INF][IMUSensor ] Initialised – accel ±2g  gyro ±250dps  sample rate 100Hz
[INF][Main      ] Connecting to WiFi...
[INF][Network   ] Connected – IP: 192.168.1.42  RSSI: -58 dBm
[INF][Collector ] Place device on a flat surface, then press the button to
calibrate.
```

The OLED then shows the **CALIBRATION** prompt screen (see below).

Step-by-Step: Calibrating and Collecting Data

- ① Power on – OLED shows "CALIBRATION" prompt screen.
- ② Place the device on a stable, flat, level surface.
- ③ Press the button once to start calibration.
The device collects 200 IMU samples (~2 s) to measure sensor bias.
Keep the device completely still during this time.
OLED shows a progress bar – do not touch the device.
- ④ OLED briefly shows "IMU calibrated!" and then switches to the IDLE screen.
The IMU is now zeroed – gyro drift and accelerometer bias are removed.
- ⑤ Press the button briefly (short press) to cycle through the activity labels:
standing → walking → running → falling → standing → ...
The label updates on the OLED immediately after each press.
- ⑥ Select the correct label for the motion you are about to perform.
- ⑦ Start a continuous recording session – choose either method:
 - Hold the button for 2 seconds (OLED shows countdown, then switches to RECORDING)
 - Or type `start` in the serial monitor and press Enter
- ⑧ After the long-press, a 2-second countdown appears on the OLED (2 → 1).

Get into position during this window – recording starts at zero.

- ⑨ Perform the labelled activity for 10 seconds.
The LED blinks rapidly and a progress bar fills on the OLED.
- ⑩ The device automatically uploads the window to Edge Impulse, shows UPLOAD OK or UPLOAD FAILED briefly, then immediately starts the NEXT 10-second window – no button press needed.
The cycle repeats indefinitely, uploading each window as it completes.
- ⑪ To end the session: press the button OR type `stop` in the serial monitor.
The current window finishes, uploads, and the OLED returns to IDLE.
You can then change the label (step ⑤) or start a new session (step ⑦).

Note: Calibration only runs once at startup. If you move the device significantly and want to recalibrate, press the **RESET** button to restart.

Countdown only appears once — at the very start of a session. Subsequent windows begin immediately after each upload so data collection is uninterrupted.

To stop the session mid-window: press the button or type `stop`. The firmware uploads however many samples were captured in the current window, then returns to IDLE.

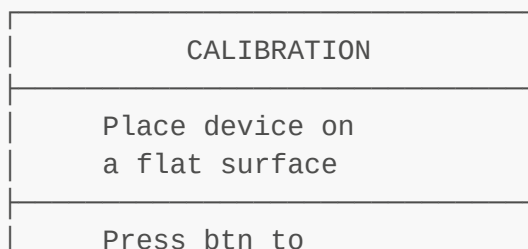
Tips for Good Data

- Let the device run for several **continuous minutes per label** — it uploads each 10-second window automatically, so you accumulate a large dataset without stopping
- Keep the sensor orientation consistent across all recordings of the same activity
- Perform the activity naturally — the model learns from real motion patterns
- Use the 2-second pre-recording countdown (first window only) to get into position
- If an upload fails, the serial monitor shows the error code; the session continues with the next window on the next attempt
- To switch labels, press the button to end the session, cycle the label with short presses, then long-press again to start a new continuous session

Understanding the Display

Each screen tells you exactly what the device is doing and what to do next.

CALIBRATION — Waiting for User Confirmation

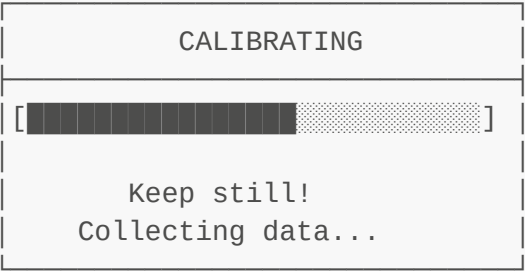


← centered state label (inverted bar)

```
calibrate IMU
```

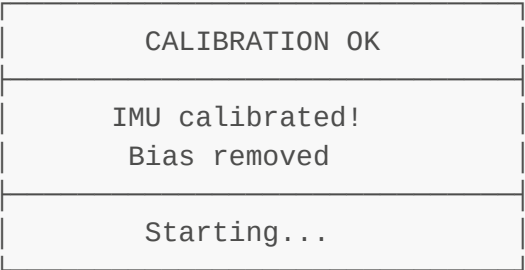
What to do: Set the device on a flat, still surface. Then press the button once.

CALIBRATING — Collecting Bias Samples



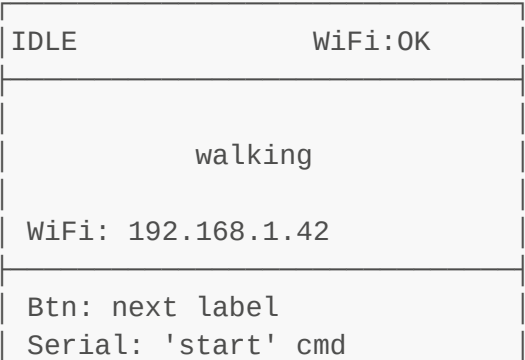
What to do: Keep the device completely still until the screen changes.

CALIBRATION OK — Complete



What to do: Nothing — the device transitions to IDLE automatically after 1.5 s.

IDLE — Waiting for Input



GET READY	walking
Recording in:	
2	

```
| REC                                walking
```

```
[ ██████████░░░░░░░░░░░░░░░░ ]
```

```
collected)
```

```
| Samples: 700 / 1000
```

```
| Time left: 3s
```

```
| Short: stop & upload
```

UPLOADING
Label: walking
Samples: 1000
Sending data to

Edge Impulse...

What to do: Wait — this takes 2–10 seconds depending on your WiFi speed.

UPLOAD OK — Success

UPLOAD OK	← success header
Label: walking Samples: 1000	
Sent to Edge Impulse! Returning to IDLE...	

What to do: Nothing — the device returns to IDLE automatically after 2.5 s.

UPLOAD FAILED — Error

UPLOAD FAILED	← failure header
Label: walking Samples: 1000	
ERR_HTTP See serial monitor	← error code

What to do: Open the serial monitor for the full error message (see Troubleshooting).

ERROR — Hardware Problem

!! SYSTEM ERROR !!	← inverted error header
IMU not detected Check: SDA→D4, SCL→D5	← error description
Check wiring + reset	

What to do: Check wiring, then press **RESET** on the XIAO.

Serial Monitor Commands

Open the serial monitor (115200 baud) to control the device from your computer. Type a command and press **Enter**.

Command	What it does
label:walking	Set the active label (standing, walking, running, falling)
start	Begin a continuous recording session (must be in IDLE; windows repeat automatically)
stop	End the continuous session: finish current window, upload it, return to IDLE
status	Print current state, label, and WiFi info
selftest	Read 10 IMU samples — acceleration should be ~9.81 m/s²
imuconfig	Print all MPU6050 register values (for debugging)
help	List all available commands

Example Session

```
> label:running
[INF][Collector] Label set to "running"

> start
[INF][Collector] Continuous recording starting in 2 s – label: "running"
(press button to stop)
[INF][Collector] Window 1 started – label: "running"  duration: 10s  rate:
100Hz
[INF][Collector]    [running] 100 samples – 9s remaining
[INF][Collector]    [running] 200 samples – 8s remaining
...
[INF][Collector] Collection complete – 1000 samples captured
[INF][Collector] Uploading to Edge Impulse (label: "running")...
[INF][Collector] Upload complete
[INF][Collector] Continuous session: auto-starting next window
[INF][Collector] Next window started – label: "running"  duration: 10s
(press button to stop)
[INF][Collector]    [running] 100 samples – 9s remaining
...

> stop
[INF][Collector] Session stopped – uploading N samples then returning to
IDLE
[INF][Collector] Upload complete
```

Note: The `start` serial command skips the pre-recording countdown and begins capturing immediately. Each subsequent window also starts without a countdown.

LED Status Guide

The onboard LED gives instant feedback without needing to watch the screen.

LED Pattern	Meaning
Single 50 ms pulse every 3 s	IDLE — device is alive and waiting for a command
Brief flash on button press	Label cycled — the OLED will confirm the new label
Single flash before countdown	Recording countdown started — get into position
Rapid on/off (every 5 samples)	RECORDING — collecting data
Solid ON	UPLOADING — sending to Edge Impulse
3 slow blinks	Upload succeeded
6 rapid blinks	Upload failed — check serial monitor
Continuous rapid blink (on boot)	Fatal error — IMU not detected

Troubleshooting

IMU Not Detected

```
[ERR][IMUSensor] WHO_AM_I mismatch: expected 0x68, got 0xFF
```

Check	How to fix
SDA/SCL wiring	D4 = SDA, D5 = SCL on the XIAO
Power	MPU6050 VCC must be 3.3 V (not 5 V)
AD0 pin	Connect AD0 to GND to select I2C address 0x68
Cable quality	Try shorter wires; add 4.7 kΩ pull-ups on SDA/SCL if signal is noisy

WiFi Connection Failed

```
[ERR][Network] Connection failed after 15000 ms
```

Check	How to fix
Credentials	Verify <code>WIFI_SSID</code> and <code>WIFI_PASSWORD</code> in <code>Config.h</code>

Check	How to fix
Frequency band	XIAO supports 2.4 GHz only — disable 5 GHz-only mode on your router
Signal strength	Move the board closer to the router for initial testing

Upload Failed — HTTP Error

```
[ERR][EIClient] Server rejected upload (HTTP 401)
```

HTTP code	Meaning	Fix
401	Wrong API key	Copy the key from Edge Impulse Studio → Dashboard → Keys
413	Payload too large	Reduce <code>COLLECTION_DURATION_S</code> or increase <code>JSON_PAYLOAD_MAX_BYTES</code> in <code>Config.h</code>

Payload Buffer Overflow

```
[ERR][EIClient] Payload buffer overflow! Increase JSON_PAYLOAD_MAX_BYTES in Config.h
```

The buffer size formula is `400 + (MAX_SAMPLES × 72)`. With the default 10-second window this is `400 + (1000 × 72) = 72 400` bytes. Increase `JSON_PAYLOAD_MAX_BYTES` in `Config.h` if you see this error.

Port Not Found (Windows)

- Install the **CH343** USB-serial driver
- Try a different cable — many USB-C cables are **charge-only**
- Check Device Manager for a yellow warning icon on the COM port

Code Deep Dive

This section is for students who want to understand how the firmware is structured and which C++ concepts appear in each file.

Project Structure

```
plastiwatch_collector/  
├─ platformio.ini           # Build configuration: board, framework,
```

```

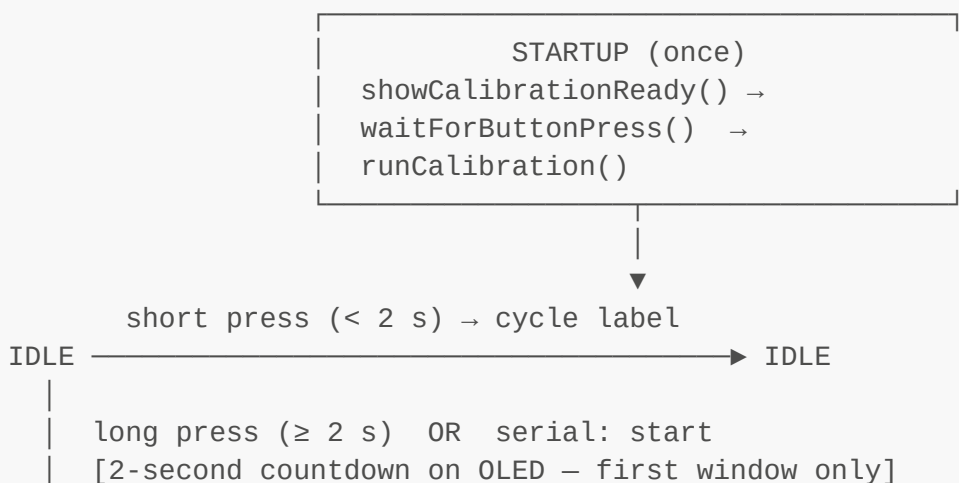
libraries
└─ src/
    │   └─ main.cpp           # Entry point – setup() and loop()
    │   └─ config/
    │       └─ Config.h       # ALL constants live here; edit before
flashing
    │   └─ core/
    │       └─ Logger.h       # Leveled serial logger (header-only,
static)
    │       └─ SampleBuffer.h # Fixed-capacity IMU sample storage
(header-only)
    │   └─ imu/
    │       └─ IMUSensor.h    # MPU6050 driver interface
    │       └─ IMUSensor.cpp  # MPU6050 driver – reads raw I2C bytes →
physical units
    │   └─ network/
    │       └─ NetworkManager.h # WiFi manager interface
    │       └─ NetworkManager.cpp # WiFi connect / reconnect logic
    │   └─ uploader/
    │       └─ EdgeImpulseClient.h # HTTPS uploader interface
    │       └─ EdgeImpulseClient.cpp # Builds JSON payload, POSTs to Edge
Impulse
    │   └─ display/
    │       └─ OLEDDisplay.h    # SSD1306 display driver interface
    │       └─ OLEDDisplay.cpp  # One draw method per application state
    │   └─ collector/
    │       └─ DataCollector.h   # State machine interface
    │       └─ DataCollector.cpp # FSM: IDLE → COLLECTING → UPLOADING →
IDLE

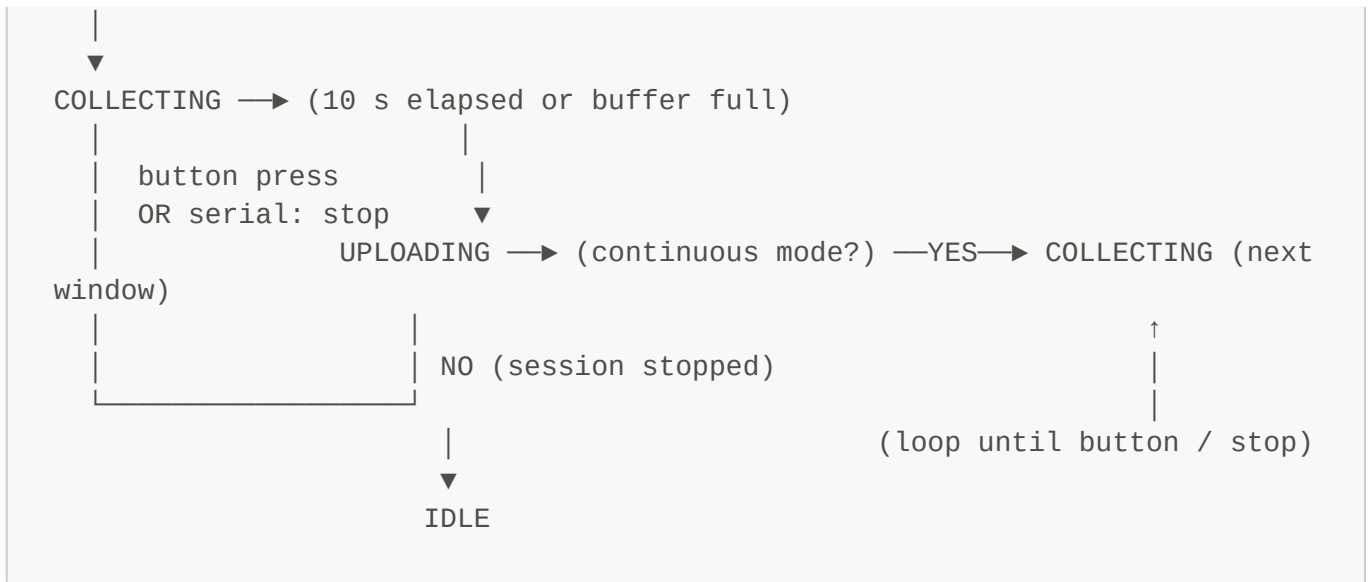
```

Key design rule: every file reads constants only from `Config.h`. To change a setting, you edit exactly one file.

How the Application Works: State Machine

`DataCollector` is a **Finite State Machine (FSM)** — the firmware is always in exactly one of these states:





`DataCollector::update()` is called on every `loop()` iteration. It:

1. Polls the label button with a two-stage debouncer (raw edge → stable edge)
2. Reads and processes any serial command
3. Dispatches to the correct state handler

IMU Calibration

At startup, before entering IDLE, the device performs a one-time bias calibration:

1. **Prompt:** OLED shows "Place device on a flat surface — press button."
2. **Confirm:** User places device flat and presses the button.
3. **Collect:** `DataCollector::runCalibration()` reads `IMU_CALIB_SAMPLES` (200) samples at 100 Hz (~2 s) while the device is stationary.
4. **Compute:** Mean values are calculated for all six axes.
 - Accel X, Y: offsets = mean (should be ≈ 0 → removes lateral bias)
 - Accel Z: offset = mean - 9.80665 (removes bias while preserving gravity)
 - Gyro X, Y, Z: offsets = mean (should be ≈ 0 → removes zero-rate drift)
5. **Apply:** `IMUSensor::setOffsets()` stores the offsets; every subsequent `readSample()` call subtracts them automatically.
6. **Result:** OLED shows "IMU calibrated!" for 1.5 s, then transitions to IDLE.

The calibration corrects for manufacturing tolerances and temperature-related drift. To recalibrate, press the **RESET** button to restart the device.

Module Descriptions

`src/config/Config.h` — Central Configuration

One header of `#define` constants — the only file you need to edit before flashing.

Key groups of settings:

- **WiFi:** SSID, password, connection timeout

- **Edge Impulse:** API key, device name, ingestion endpoint
- **MPU6050:** I2C pins, full-scale ranges, digital filter, sample rate
- **Collection:** recording duration (10 s), window size (1000 samples), pre-recording countdown delay (2 s), calibration sample count (200)
- **Display:** OLED I2C address, screen dimensions, result dwell time

src/core/Logger.h — Serial Logger (header-only)

A static-only utility class that formats log messages with a level tag and a module tag:

```
[INF][Collector      ] Recording started — label: "walking"
[ERR][IMUSensor      ] WHO_AM_I mismatch: expected 0x68, got 0xFF
[DBG][EIClient        ] Payload built: 72412 bytes
```

Log levels: `DEBUG < INFO < WARNING < ERROR < NONE`

Verbose debug logging is enabled by building with `-DPLASTIWATCH_DEBUG=1` (the `seeed_xiao_esp32c3_debug` environment does this automatically).

src/core/SampleBuffer.h — IMU Sample Storage (header-only)

A fixed-size array wrapper for one collection window. Uses a **statically allocated** array — no heap allocation, no `std::vector`.

```
struct IMUSample {
    float accX, accY, accZ; // Calibrated acceleration [m/s²]
    float gyrX, gyrY, gyrZ; // Calibrated angular velocity [deg/s]
};
```

Key methods: `push()`, `count()`, `isFull()`, `isEmpty()`, `clear()`, `operator[]`, `fillRatio()`, `intervalMs()`.

src/imu/IMUSensor.h/.cpp — MPU6050 Driver

Communicates with the MPU6050 directly over the Arduino `Wire` (I2C) library — no third-party sensor library required.

Initialization sequence (`begin()`):

1. Start I2C bus at 400 kHz
2. Read `WHO_AM_I` register — must return `0x68` (identity check)
3. Wake the device, select stable PLL clock source
4. Configure full-scale ranges and digital low-pass filter (DLPF)
5. Pre-compute float scale factors for fast conversion in the read loop

Reading data (`readSample()`):

- Burst-reads 14 bytes from register `0x3B` in one I2C transaction
- Reassembles big-endian 16-bit integers using a local lambda
- Multiplies by scale factors to produce m/s^2 and deg/s
- **Subtracts calibration offsets** (set to 0 until `setOffsets()` is called)

Scale factor formulas:

```
accelScale = 9.80665 / (16384 / FS_G)      // raw ADC → m/s²
gyroScale  = 1.0 / (131 / (FS_DPS / 250)) // raw ADC → deg/s
```

Calibration (`setOffsets()`): Stores per-axis bias values that are subtracted inside every `readSample()` call. Called once by `DataCollector::runCalibration()` during startup.

src/network/NetworkManager.h/.cpp — WiFi Manager

A static-only class (cannot be instantiated) wrapping the ESP32 WiFi library.

Method	Behaviour
<code>connect()</code>	Blocking connect with progress dots; times out after 15 s
<code>ensureConnected()</code>	Non-blocking guard; retries at most once per 5 s
<code>isConnected()</code>	True when an IP address is assigned
<code>localIP() / rssi()</code>	Diagnostics — IP address and signal strength

src/uploader/EdgeImpulseClient.h/.cpp — HTTPS Uploader

Serialises a `SampleBuffer` into Edge Impulse's **Data Acquisition JSON** format and POSTs it over HTTPS.

Payload skeleton:

```
{
  "protected": { "ver": "v1", "alg": "none", "iat": 0 },
  "signature": "0000...0000",
  "payload": {
    "device_name": "plastiwatch-001",
    "interval_ms": 10.0,
    "sensors": [
      { "name": "accX", "units": "m/s2" }, ...
    ],
    "values": [
      [-0.12, 9.78, 0.05, 1.20, -0.30, 0.10],
      ...1000 rows total...
    ]
  }
}
```

```
    }  
}
```

The internal `PayloadBuilder` class writes into a pre-allocated `char[]` buffer with overflow detection — no `std::string`, no dynamic resizing.

Upload result codes:

Code	Meaning
OK	Success (HTTP 200/201)
ERR_NO_WIFI	Not connected to WiFi
ERR_NO_SAMPLES	Buffer is empty
ERR_ALLOC	Heap allocation failed
ERR_OVERFLOW	JSON exceeded buffer size
ERR_HTTP	Server returned a non-2xx response
ERR_TIMEOUT	Connection or request timed out

src/display/OLEDDisplay.h/.cpp — OLED Driver

Wraps the Adafruit SSD1306 library. Exposes one draw method per application state. All methods are safe no-ops if the display is not connected (`begin()` returned false).

Screen methods:

Method	When shown
<code>showBoot()</code>	Power-on splash
<code>showCalibrationReady()</code>	Waiting for user to confirm calibration
<code>showCalibrating(progress)</code>	Calibration sample collection in progress
<code>showCalibrationDone()</code>	Calibration complete — 1.5 s dwell
<code>showIdle(label, wifi, ip)</code>	IDLE state — awaiting user input
<code>showRecordingCountdown(label, s)</code>	Pre-recording countdown (one call per second)
<code>showCollecting(label, n, total, t)</code>	Recording in progress
<code>showUploading(label, n)</code>	HTTP upload in progress
<code>showUploadResult(ok, label, n, err)</code>	Upload result — 2.5 s dwell
<code>showError(msg)</code>	Fatal hardware error

Private helpers:

- `printCentered(text, y, size)` — centres text horizontally at a given y coordinate
- `drawHeaderBar(left, right)` — draws the inverted (white-fill, black-text) header bar
- `drawProgressBar(x, y, w, h, ratio)` — draws a filled progress bar with border

`src/collector/DataCollector.h/.cpp` — State Machine

The application's core logic. Orchestrates IMU reading, button input, serial commands, display updates, and upload sequencing.

`begin()` startup sequence:

1. Configure GPIO pins
2. Show calibration prompt on OLED
3. Block until user presses the button (`waitForButtonPress()`)
4. Run IMU calibration (`runCalibration()`) — 200 samples, ~2 s
5. Show calibration result briefly
6. Show IDLE screen

`update()` is called every loop iteration and:

1. Polls the label button (GPIO3) with a two-stage debouncer
2. Polls the serial port for text commands
3. Dispatches to `handleIdle()`, `handleCollecting()`, or `handleUploading()`

Button behaviour:

- **IDLE — Short press** ($< \text{BTN_LONG_PRESS_MS} = 2000 \text{ ms}$) → cycle the active label
- **IDLE — Long press** ($\geq 2 \text{ s}$, then release) → show countdown, start continuous session
- **COLLECTING — Any press** → end continuous session; current window is uploaded, then the device returns to IDLE (no new window starts)
- The debouncer ignores transitions shorter than `BTN_DEBOUNCE_MS` (50 ms), preventing contact bounce from registering as multiple presses
- All actions fire on **button release** (rising edge); press-start time is stamped on the falling edge to measure hold duration

Continuous recording session: `startRecording()` sets `m_continuousMode = true` and shows a per-second countdown (first window only). After each window, `handleUploading()` checks the flag: if still set it calls `restartRecording()` (no countdown) immediately; if cleared (user stopped the session) it transitions to IDLE. The flag is cleared by `stopAndUpload()`, which is called on button press during COLLECTING or on the serial `stop` command.

`src/main.cpp` — Entry Point

Standard Arduino structure, kept intentionally minimal:

```
setup()  
|
```

└ Serial.begin(115200)	set up serial monitor
└ Logger::setLevel(...)	INFO or DEBUG depending on build flags
└ display.begin() attached)	initialise OLED (safe to skip if not
└ imu.begin())	initialise MPU6050 – HALT on failure
└ NetworkManager::connect() offline	connect to WiFi – warn and continue
└ collector.begin() IDLE screen	calibration sequence → show initial
loop() └ collector.update() iteration	non-blocking, returns immediately every iteration

C++ Concepts Used

This project was designed as a teaching tool. Each concept is used deliberately and appears in the same place every time.

Concept	Where Applied	Why
enum class (scoped enum)	LogLevel, CollectorState, UploadResult	Type-safe; can't accidentally compare states to integers
Static-only / utility classes	Logger, NetworkManager, EdgeImpulseClient	No instance needed — constructor is deleted
Constructor member initializer lists	DataCollector, IMUSensor	Preferred C++ initialization style
constexpr	SampleBuffer::CAPACITY, IMU register addresses	Values resolved at compile time — zero runtime cost
explicit constructor	DataCollector(IMUSensor& imu)	Prevents accidental implicit conversions
Reference member	IMUSensor& m_imu in DataCollector	Dependency injection without ownership
Lambda	toInt16 in readSample()	Short, inline helper that captures nothing
Anonymous namespace	PayloadBuilder in EdgeImpulseClient.cpp	Hides implementation details from the linker
new (std::nothrow)	Heap allocation in EdgeImpulseClient	Returns nullptr on failure instead of throwing
static local variables	lastHeartbeatMs / ledOn in handleIdle()	Persist across function calls without being global

Concept	Where Applied	Why
<code>#pragma once</code>	All header files	Include guard — simpler than <code>#ifndef</code> guards
<code>inline static member</code>	<code>Logger::s_level</code>	C++17 feature — lets you define a static member inside a header
Variadic functions (<code>va_list</code>)	<code>Logger::infof()</code> , <code>PayloadBuilder::appendf()</code>	Printf-style formatting with variable argument counts
<code>reinterpret_cast</code>	<code>char* → uint8_t*</code> in <code>EdgeImpulseClient</code>	Type-safe cast required by the HTTPClient API
<code>static_cast</code>	Numeric conversions throughout	Explicit, readable type conversions
Operator overloading	<code>SampleBuffer::operator[]</code>	Natural array-style element access
<code>switch</code> on scoped enum	State dispatch in <code>update()</code>	Clean, exhaustive state handling
Array member initializer	<code>m_accelOffsets{0,0,0}</code> in <code>IMUSensor</code>	C++11 brace-initialization of array members

Build Configuration (`platformio.ini`)

```
[env:seeed_xiao_esp32c3]
board           = seeed_xiao_esp32c3
platform       = espressif32
framework     = arduino
upload_speed   = 921600
build_flags    = -std=gnu++17           ← enables C++17 (inline static
members, etc.)
lib_deps =
  adafruit/Adafruit SSD1306@^2.5.9
  adafruit/Adafruit GFX Library@^1.11.9
```

PlatformIO downloads all library dependencies automatically on the first build — no manual library installation required.

Target board: Seeed Studio XIAO ESP32C3 — Framework: Arduino — Build system: PlatformIO